



Escuela de Administración de Tecnologías de Información

Proyecto 1

Elaborado por

Brandon Rojas Garita

Curso

Bases de datos

Profesora

Jacqueline Solís Céspedes

Septiembre 2026

Cartago, Costa Rica

Tabla de contenido

1. Selección de los módulos ampliados	2
2. Requerimientos de diseño y arquitectura.....	3
3. Documentación y trazabilidad.....	3
4. Detalle de modelos por módulo	4
4.1 Gestión de Ubicaciones (RF-08, RF-09, RF-10).....	4
4.2 Disponibilidad de Usuarios (RF-11, RF-12).....	5
4.3 Tareas Asociadas a Eventos (RF-15, RF-16, RF-17).....	6
5. Evidencias Visuales de la Interfaz Gráfica (GUI)	7

Índice de Figuras

Figura 1 . Modelo relacional de Agenda	4
Figura 2 - Modelo conceptual de Ubicaciones	5
Figura 3 - Modelo conceptual de Disponibilidad	5
Figura 4 - Modelo conceptual de Tareas	6
Figura 5 - Captura de la pestaña Usuarios	7
Figura 6 - Captura de la pestaña Categorías	7
Figura 7 - Captura de la pestaña Ubicaciones	8
Figura 8 - Captura de la pestaña Eventos	8
Figura 9 - Validación de traslape de horario	9
Figura 10 - Captura de la pestaña Disponibilidad	9
Figura 11 - Captura de la pestaña Tareas	10

1. Selección de los módulos ampliados

Para la ampliación del sistema se seleccionaron tres de los cuatro módulos disponibles: Ubicaciones, Disponibilidad de Usuarios y Tareas Asociadas a Eventos. Esta selección se hizo porque los tres módulos permiten enriquecer el sistema con funcionalidades de negocio claras y bien delimitadas, manteniendo una complejidad manejable dentro del tiempo disponible. Se dejó fuera el módulo de Eventos Recurrentes, ya que su implementación requiere lógica adicional de generación automática de fechas que excede el alcance práctico de esta entrega.

2. Requerimientos de diseño y arquitectura

A partir de los tres módulos seleccionados, la ampliación se realizó de forma progresiva y estructurada, siguiendo los mismos cuatro niveles de diseño que se usaron para el núcleo original del sistema:

- **Modelo Conceptual (Notación de Chen):** representación gráfica de las nuevas entidades, sus atributos principales, y las cardinalidades que las conectan con las entidades ya existentes (Usuarios, Eventos).
- **Modelo Lógico:** traducción de cada esquema conceptual a un modelo relacional, declarando claves primarias (PK) y claves foráneas (FK).
- **Modelo Físico:** especificación técnica para PostgreSQL, incluyendo la creación de tablas, tipos de datos, restricciones (CHECK, NOT NULL) y reglas de eliminación en cascada donde corresponde.
- **Modelo de Implementación (Script SQL):** artefacto único y consolidado (script.sql) que materializa el modelo físico completo, permitiendo restaurar toda la base de datos en un solo paso.

Cada uno de estos niveles se documenta en detalle, módulo por módulo, en la Sección 4 de este documento.

3. Documentación y trazabilidad

La trazabilidad del desarrollo se organizó por fases (conceptual, lógica, física y de interfaz gráfica), registrando el estado de cada tarea a medida que se completaba. La siguiente matriz resume el estado actual de los tres módulos ampliados:

Matriz de trazabilidad y estado de implementación

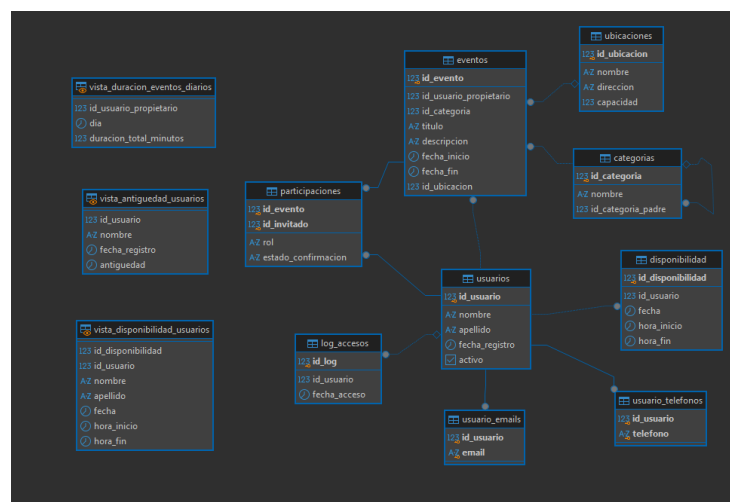
Req.	Fase	Tarea	Estado
RF-08	Modelo Conceptual	Definir la entidad ubicaciones y su relación con eventos.	Completado
RF-08	Modelo Lógico	Estructurar la tabla ubicaciones (id_ubicacion PK, nombre, direccion, capacidad).	Completado
RF-09	Modelo Físico	Agregar columna id_ubicacion (FK) a la tabla eventos.	Completado
RF-10	Modelo Físico	Crear trigger que bloquea traslapes de horario en una misma ubicación.	Completado
RF-08 a RF-10	Interfaz Gráfica	Pestaña Ubicaciones (CRUD) y selector de ubicación en Eventos.	Completado

Req.	Fase	Tarea	Estado
RF-11	Modelo Conceptual	Definir la entidad disponibilidad y su relación con usuarios.	Completado
RF-11	Modelo Lógico	Estructurar la tabla disponibilidad (id_disponibilidad PK, id_usuario FK, fecha, horas).	Completado
RF-12	Modelo Físico	Crear vista para consultar bloques de disponibilidad por usuario.	Completado
RF-11 y RF-12	Interfaz Gráfica	Pestaña Disponibilidad (CRUD).	Completado
RF-15	Modelo Conceptual	Definir la entidad tareas y sus relaciones con eventos y usuarios.	Completado
RF-15	Modelo Lógico	Estructurar la tabla tareas (id_tarea PK, titulo, prioridad, estado, fecha_limite, FKs).	Completado
RF-16	Modelo Físico	Restricciones de prioridad/estado y eliminación en cascada con eventos.	Completado
RF-17	Modelo Físico	Crear vista que calcula dinámicamente las tareas vencidas.	Completado
RF-15 a RF-17	Interfaz Gráfica	Pestaña Tareas (CRUD con selectores de evento y responsable).	Completado

4. Detalle de modelos por módulo

El siguiente es el modelo relacional automáticamente por DBeaver a partir del esquema físico real de la base de datos, mostrando las diez tablas del sistema: las siete originales más las tres incorporadas en esta ampliación junto con sus claves primarias, claves foráneas y las relaciones entre ellas:

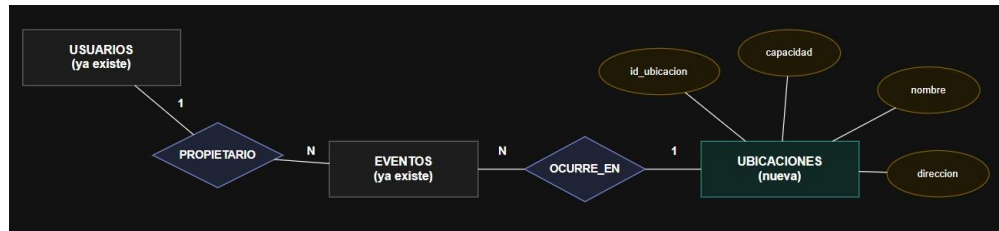
Figura 1 . Modelo relacional de Agenda



4.1 Gestión de Ubicaciones (RF-08, RF-09, RF-10)

La entidad Ubicaciones representa los recintos físicos donde pueden ocurrir los eventos. Cada ubicación puede alojar muchos eventos, pero cada evento ocurre en una única ubicación (relación 1 a N).

Figura 2 - Modelo conceptual de Ubicaciones



A nivel físico, se agregó la columna id_ubicacion como llave foránea en la tabla eventos, y se implementó un disparador (trigger) que impide guardar un evento si su horario se cruza con otro evento ya registrado en la misma ubicación:

```

CREATE TABLE ubicaciones (
    id_ubicacion SERIAL PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    direccion VARCHAR(150),
    capacidad INT CHECK (capacidad > 0)
);

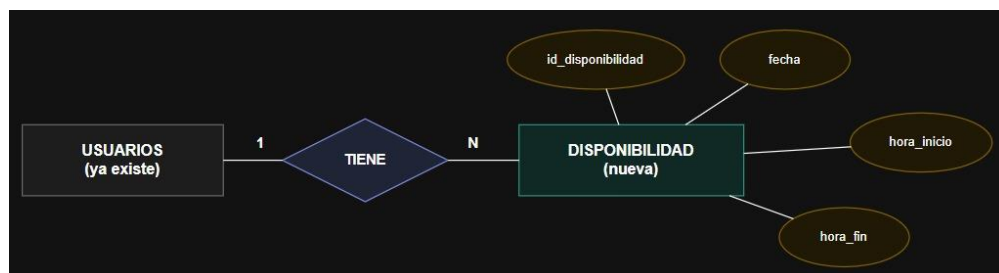
ALTER TABLE eventos
    ADD COLUMN id_ubicacion INT REFERENCES ubicaciones(id_ubicacion);

CREATE TRIGGER trg_evitar_traslape_ubicacion
BEFORE INSERT OR UPDATE ON eventos
FOR EACH ROW EXECUTE FUNCTION evitar_traslape_ubicacion();
  
```

4.2 Disponibilidad de Usuarios (RF-11, RF-12)

La entidad Disponibilidad registra los bloques de horario en que un usuario está libre. Un usuario puede tener muchos bloques de disponibilidad (relación 1 a N con Usuarios).

Figura 3 - Modelo conceptual de Disponibilidad



Además, se creó una vista de apoyo que combina la disponibilidad con los datos del usuario, para facilitar su consulta al momento de agendar un evento:

```

CREATE TABLE disponibilidad (
    id_disponibilidad SERIAL PRIMARY KEY,
    id_usuario INT NOT NULL REFERENCES usuarios(id_usuario) ON DELETE CASCADE,
    fecha DATE NOT NULL,
  
```

```

    hora_inicio TIME NOT NULL,
    hora_fin TIME NOT NULL,
    CONSTRAINT check_horas_disponibilidad CHECK (hora_fin > hora_inicio)
);

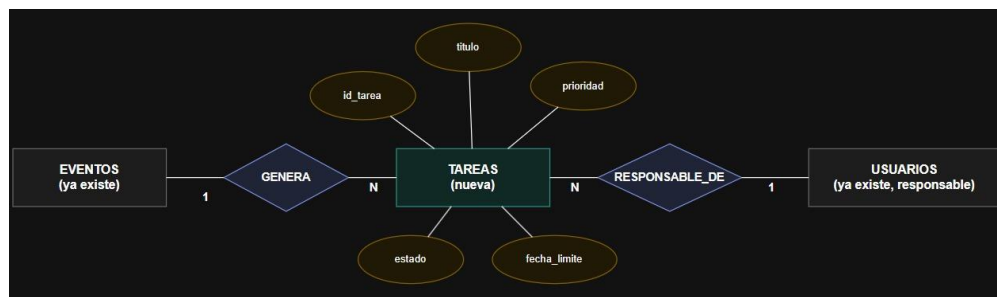
CREATE VIEW vista_disponibilidad_usuarios AS
SELECT d.id_disponibilidad, d.id_usuario, u.nombre, u.apellido,
       d.fecha, d.hora_inicio, d.hora_fin
FROM disponibilidad d
JOIN usuarios u ON u.id_usuario = d.id_usuario
ORDER BY d.fecha, d.hora_inicio;

```

4.3 Tareas Asociadas a Eventos (RF-15, RF-16, RF-17)

La entidad Tareas representa subtareas operativas ligadas a un evento, con un responsable opcional. Un evento puede tener muchas tareas, y un usuario puede ser responsable de muchas tareas.

Figura 4 - Modelo conceptual de Tareas



Siguiendo el mismo principio usado para calcular la antigüedad de un usuario, se creó una vista que calcula en tiempo real qué tareas están vencidas, sin almacenar ese estado de forma física:

```

CREATE TABLE tareas (
    id_tarea SERIAL PRIMARY KEY,
    titulo VARCHAR(100) NOT NULL,
    descripcion TEXT,
    prioridad VARCHAR(10) NOT NULL DEFAULT 'media'
        CHECK (prioridad IN ('baja', 'media', 'alta')),
    estado VARCHAR(20) NOT NULL DEFAULT 'pendiente'
        CHECK (estado IN ('pendiente', 'en progreso', 'completada')),
    fecha_limite DATE,
    id_evento INT NOT NULL REFERENCES eventos(id_evento) ON DELETE CASCADE,
    id_usuario_responsable INT REFERENCES usuarios(id_usuario)
);

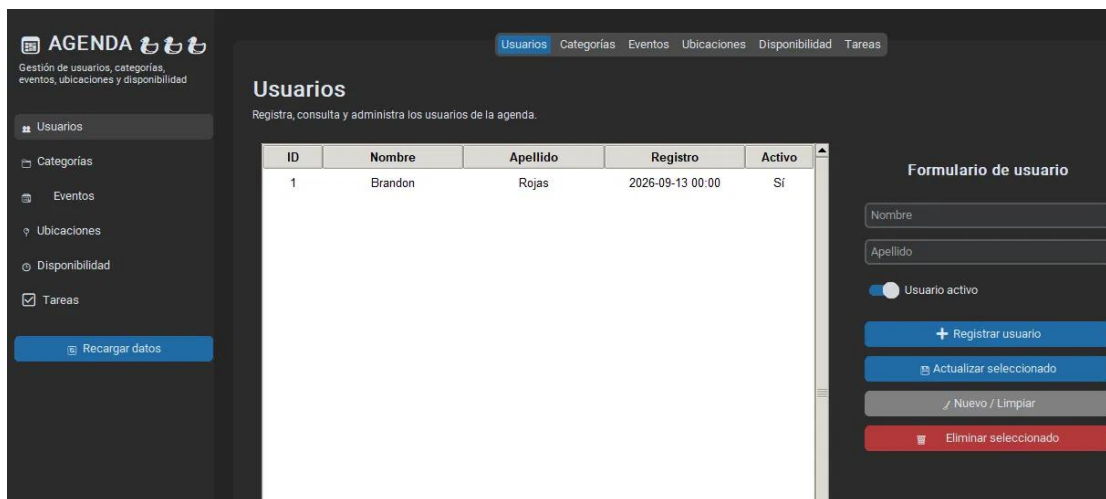
CREATE VIEW vista_tareas_vencidas AS
SELECT t.id_tarea, t.titulo, t.prioridad, t.estado, t.fecha_limite,
       e.titulo AS evento, u.nombre, u.apellido
FROM tareas t
JOIN eventos e ON e.id_evento = t.id_evento
LEFT JOIN usuarios u ON u.id_usuario = t.id_usuario_responsable
WHERE t.fecha_limite < CURRENT_DATE
      AND t.estado <> 'completada';

```

5. Evidencias Visuales de la Interfaz Gráfica (GUI)

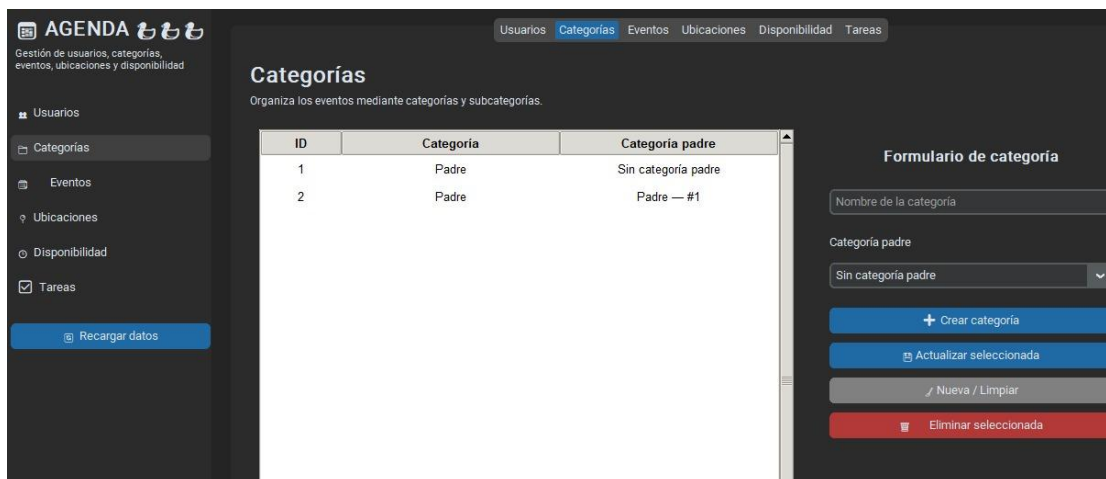
A continuación, se presentan capturas de pantalla de la aplicación de escritorio, con el propósito de demostrar la correcta integración entre la capa visual (desarrollada en Python) y el motor de base de datos PostgreSQL. Cada imagen incluye una breve explicación de sus componentes, evidenciando los formularios CRUD, los selectores dinámicos, y su conexión en tiempo real con la base de datos.

Figura 5 - Captura de la pestaña Usuarios



Se muestra el formulario CRUD de Usuarios junto con la tabla que refleja en tiempo real los registros almacenados en la tabla usuarios de PostgreSQL. El formulario permite registrar, actualizar y eliminar usuarios, confirmando la integración entre la capa visual y el motor de base de datos.

Figura 6 - Captura de la pestaña Categorías



Se observa el selector dinámico "Categoría padre", el cual se llena automáticamente con las categorías ya existentes en la base de datos. En la tabla se aprecia la categoría "Padre - #2" registrada como subcategoría de "Padre - #1", demostrando la relación jerárquica recursiva implementada en la tabla categorías.

Figura 7 - Captura de la pestaña Ubicaciones

AGENDA

Gestión de usuarios, categorías, eventos, ubicaciones y disponibilidad

Usuarios | Categorías | Eventos | **Ubicaciones** | Disponibilidad | Tareas

Ubicaciones

Administra las salas y espacios donde ocurren los eventos.

ID	Nombre	Dirección	Capacidad
2	Sala 9 Norte - Torre Sur	Pueblo Parmesano	20
1	Sala A - Edificio de Consejo	Parque Placentero	60

Formulario de ubicación

Nombre (ej: Sala de Juntas A)

Dirección

Capacidad (número de personas)

+ Crear ubicación

Actualizar seleccionada

/ Nueva / Limpiar

Eliminar seleccionada

Recargar datos

Pantalla del módulo nuevo de Ubicaciones, mostrando dos ubicaciones registradas con su nombre, dirección y capacidad. El formulario valida que la capacidad ingresada cumpla con la restricción CHECK (capacidad > 0) definida a nivel de base de datos.

Figura 8 - Captura de la pestaña Eventos

AGENDA

Gestión de usuarios, categorías, eventos, ubicaciones y disponibilidad

Usuarios | Categorías | **Eventos** | Ubicaciones | Disponibilidad | Tareas

Eventos

Programa eventos seleccionando usuarios, categorías, ubicación, fechas y horas.

ID	Propietario	Categoría	Ubicación	Título
1	Brandon Rojas — #1	Padre — #1	Sala A - Edificio de C	Cumpleaños

Formulario de evento

Título del evento

Propietario

Seleccione un usuario

Categoría

Seleccione una categoría

Ubicación (opcional)

Sin ubicación asignada

Inicio

2026-09-13 09:00

Fin

2026-09-13 10:00

+ Crear evento

Actualizar seleccionado

/ Nuevo / Limpiar

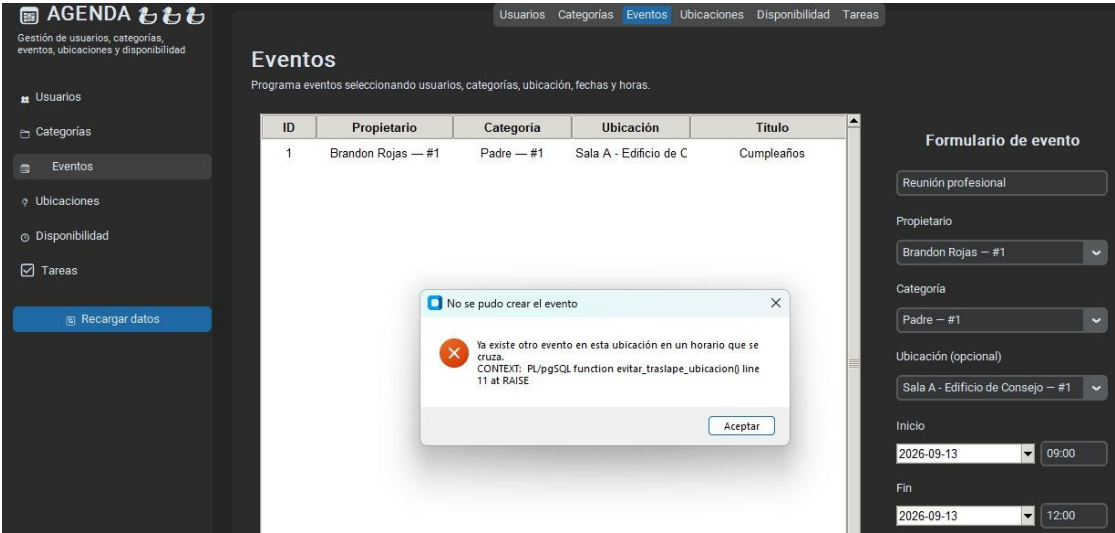
Recargar datos

APARIENCIA

System

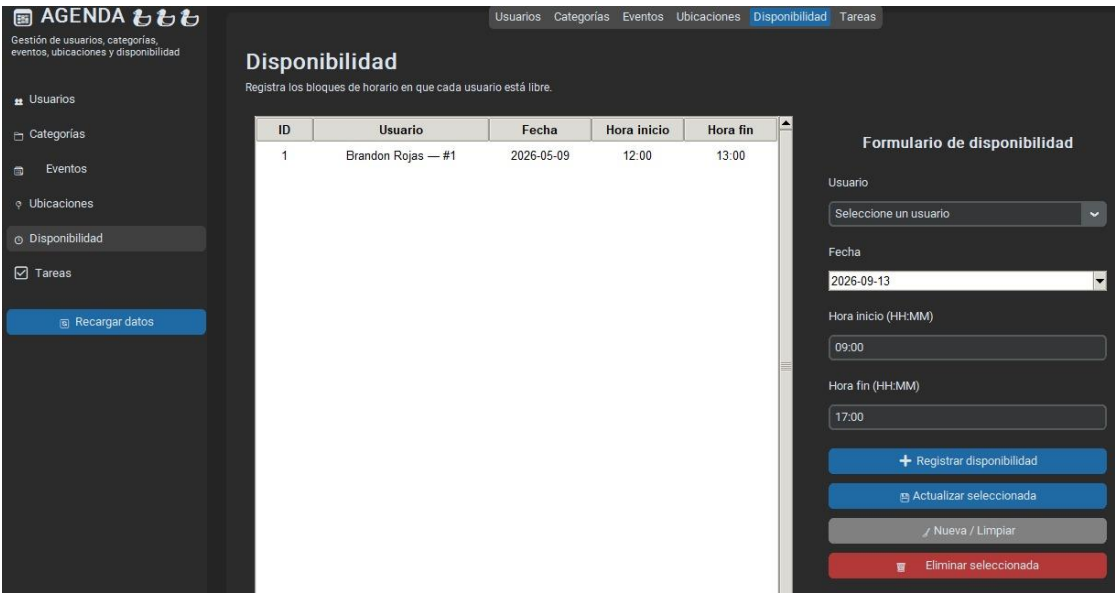
Se muestra el formulario de Eventos con sus selectores dinámicos de Propietario, Categoría y Ubicación, cada uno poblado en tiempo real desde sus respectivas tablas relacionadas. La tabla central refleja el evento ya creado, con la ubicación asignada correctamente visible en su columna.

Figura 9 - Validación de traslape de horario



Demostración en vivo de la regla de negocio implementada mediante el trigger `trg_evitar_traslape_ubicacion`: al intentar crear un segundo evento en la misma ubicación con un horario que se cruza con uno ya existente, PostgreSQL rechaza la operación y el mensaje de error generado por el disparador se muestra directamente en la interfaz gráfica.

Figura 10 - Captura de la pestaña Disponibilidad



Pantalla del módulo nuevo de Disponibilidad (RF-11), con un bloque de horario libre registrado para el usuario Brandon Rojas. El selector de Usuario se alimenta dinámicamente de la tabla usuarios, y la tabla central consulta los datos a través de la vista `vista_disponibilidad_usuarios`.

Figura 11 - Captura de la pestaña Tareas

The screenshot displays the 'Tareas' (Tasks) module within the 'AGENDA' system. The left sidebar contains navigation links for 'Usuarios', 'Categorías', 'Eventos', 'Ubicaciones', 'Disponibilidad', and 'Tareas' (selected). Below these is a 'Recargar datos' button. The main area features a table with the following data:

ID	Título	Evento	Responsable	Prioridad
1	Recibimiento invitados	Cumpleaños	Christian Rojas	media

To the right of the table is a form for creating or updating tasks. It includes dropdown menus for 'Evento' (Selecione un evento), 'Responsable (opcional)' (Sin responsable asignado), 'Prioridad' (media), and 'Estado' (pendiente). There is also a date picker for 'Fecha límite' set to '2026-09-13'. At the bottom of the form are buttons: '+ Crear tarea', 'Actualizar seleccionada', '/ Nueva / Limpiar', and 'Eliminar seleccionada'.

Pantalla del módulo nuevo de Tareas (RF-15), mostrando una tarea creada con su evento asociado, responsable, prioridad y estado. Los selectores de Evento y Responsable se llenan dinámicamente desde las tablas eventos y usuarios respectivamente, evidenciando la integración completa de los tres módulos ampliados con el núcleo preexistente del sistema.